

Assignment 3: Bridge Pattern Report

Course: ShP-2216 Software Design Patterns | 2026–2027

Instructor: Yerassyl Bekenov

Student: Dayana Zharylgapova | Group: SE-2538

Topic: Option B — Notifications System

1. Architectural Overview & Bridge Pattern Rationale

1.1 Two Independent Dimensions

The application addresses a notification delivery system that naturally varies along two orthogonal, independent dimensions:

- **Abstraction Hierarchy (Notification Types):** Represents high-level business entities and domain logic (Reminder and UrgentAlert).
- **Implementor Hierarchy (Delivery Channels):** Represents low-level transport mechanisms (EmailChannel, SmsChannel, and PushChannel).

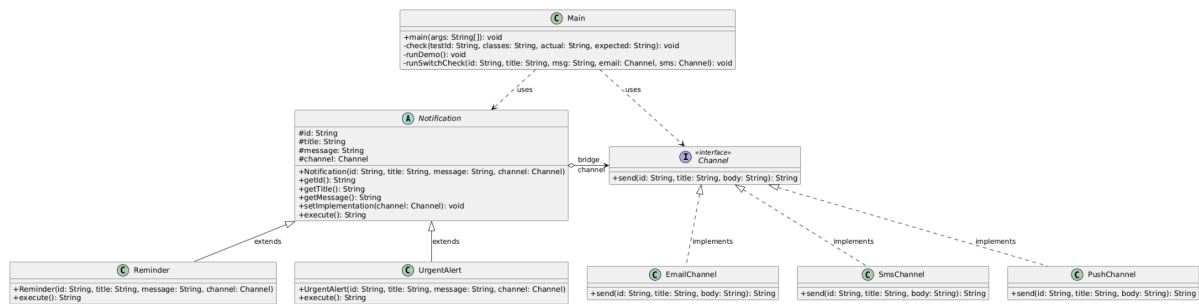
1.2 Why Bridge Pattern Fits

Without the Bridge pattern, combining M notification types with N delivery channels using traditional inheritance results in an $M * N$ class explosion (e.g., EmailReminder, SmsReminder, PushReminder, EmailUrgentAlert, SmsUrgentAlert, etc.). This leads to code duplication and tight coupling.

The Bridge pattern decouples the abstraction hierarchy from the implementor hierarchy by placing a composition reference (protected Channel channel;) inside the abstract base class Notification. This design offers three crucial benefits:

- **Decoupling:** Abstractions and implementations can evolve completely independently.
- **Runtime Switchability:** Delivery channels can be dynamically changed on an existing Notification object at runtime using setImplementation(...) without losing domain state.
- **Extensibility:** New channels (like I3 PushChannel) can be added without modifying existing classes.

2. UML Class Diagram



3. Clean Code Excerpts and Analysis

Excerpt 1: Separation of Responsibilities

```
1 public abstract class Notification {
2     protected String id;
3     protected String title;
4     protected String message;
5     protected Channel channel;
6
7     public Notification(String id, String title, String message, Channel channel) {
8         this.id = id;
9         this.title = title;
10        this.message = message;
11        this.channel = channel;
12    }
13
14    public String getId() { return id; }
15    public String getTitle() { return title; }
16    public String getMessage() { return message; }
17
18    public void setImplementation(Channel channel) { this.channel = channel; }
19
20    public String execute() { return channel.send(id, title, message); }
21 }
```

Explanation: The abstract base class **Notification** manages domain data (`id`, `title`, `message`) while delegating low-level string formatting and delivery entirely to the **Channel** interface. This adheres to the Single Responsibility Principle (SRP).

Excerpt 2: Meaningful Names

```
1 public class UrgentAlert extends Notification {
2     public UrgentAlert(String id, String title, String message, channel channel) {
3         super(id, title, message, channel);
4     }
5     @Override
6     public String execute() {
7         return channel.send(id, "[URGENT] " + title, message);
8     }
9 }
```

Explanation: Domain-specific concepts are explicitly expressed through self-documenting naming conventions (UrgentAlert, execute, [URGENT]). The intent is clear without requiring inline comments.

Excerpt 3: Small, Focused Methods

```
1 public class SmsChannel implements Channel{
2     @Override
3     public String send(String id, String title, String body) {
4         return "SMS " + id + ": " + title + " - " + body;
5     }
6 }
```

Explanation: The send method is short, cohesive, and handles a single formatting task without side effects. Small methods improve readability and testability.

Excerpt 4: Avoidance of Duplication (DRY Principle)

```
public void setImplementation(Channel channel) { this.channel = channel; }
```

Explanation: The runtime channel replacement logic and state storage are defined once in the abstract base class Notification, preventing code repetition across refined abstraction subclasses (Reminder and UrgentAlert).

Excerpt 5: Encapsulation of State

```
public String getId() { return id; }
public String getTitle() { return title; }
public String getMessage() { return message; }
```

Explanation: Domain fields are declared as protected and exposed via immutable getter methods. This prevents unauthorized state modification during runtime channel switches.

4. Extension, Trade-offs, and Pattern Comparison

4.1 Independent Extension (I3 - PushChannel)

To add PushChannel (I3) as an extension:

- 1. Created PushChannel.java implementing the Channel interface.
- 2. Added checks T6 and T7 to Main.java.
- 3. Included src/PushChannel.java in sources.txt.

Unchanged Source Files: The Notification base class, Reminder, UrgentAlert, Channel interface, EmailChannel, and SmsChannel were left completely untouched. The source diff (extension.diff) confirms that changes were strictly limited to adding the new PushChannel class and updating Main.java. This proves adherence to the Open/Closed Principle (OCP).

4.2 Trade-offs of the Bridge Pattern

Trade-off / Downside: Bridge increases overall structural complexity by introducing additional interface indirections and splitting code across multiple class hierarchies. For small systems with only one dimension of variation, Bridge introduces unnecessary overhead compared to simple inheritance.

4.3 Intent Comparison: Bridge vs. Adapter

Characteristic	Bridge Pattern	Adapter Pattern
Design Phase	Designed upfront during initial architecture.	Applied retrospectively to legacy or 3rd-party code.

Characteristic	Bridge Pattern	Adapter Pattern
Purpose	Allows abstractions and implementations to vary independently.	Makes existing, incompatible interfaces work together.
Structure	Separates abstraction from implementor via composition.	Wraps an existing adaptee to match a target interface.

5. References

1. Bekenov, Y. (2026). *Lecture 4: Bridge Pattern*. Course: ShP-2216 Software Design Patterns, Astana IT University.
2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.